

Mutation

CSCI 1101

2024-02-15

Mutability

Up until now we have been blissfully working within the portion of python where things stay the way that we left them.

The types `int`, `float`, `bool`, `str`, and `NoneType` are **immutable**: there is no way to change a value of these types once it has been created.

The `list` types are different: we can create a list and then later *change* its contents in some way.

List Index Assignment

We may put the list index operator `_[_]` on the left side of an assignment statement:

```
<list>[<index>] = <new_element>
```

This **index assignment statement** causes the value of the expression on the right to replace the current element at the specified index in the list on the left.

```
xs = [1, 2, 3]  
xs[1] = 4
```

List Slice Assignment

Similarly, we may put the list slice operator `_[_:_]` on the left side of an assignment statement:

```
<list>[<start_index>:<stop_index>] = <new_list>
```

This **slice assignment statement** causes the list on the right to replace the current slice between the specified indices in the list on the left.

```
xs = [1, 2, 3]  
xs[1:2] = [5, 6]
```

You can think of this as “cutting out” the slice specified by the indices and “splicing in” the new list in its place. This is like analog film editing.

Important cases:

- If the list spliced in is empty then this just cuts out a slice.
- If the list cut out is empty then this just splices in a slice.

Activity: List Surgery

Fill in the ?s of the following transcript with the python interactive interpreter:

```
> xs = [1, 2, 3]
```

```
> xs[?] = 4
```

```
> xs
```

```
[1, 2, 4]
```

```
> xs[?:?] = []
```

```
> xs
```

```
[2, 4]
```

```
> xs[?:?] = xs
```

```
> xs
```

```
[2, 2, 4, 4]
```

```
> xs[1:3] = ?
```

```
> xs
```

```
[2, 3, 4]
```

List Mutating Functions: Reversing

Because lists are mutable, it is important to understand when passing a list to a function can change that list in some way.

Recall the list-reversing function we met last time:

```
reversed : (list a) --> iterator a
```

This function *returns* a reversed version of the argument list, but leaves the argument list alone.

Python has another list-reversing function:

```
list.reverse : (list a) --> NoneType
```

This function *returns* nothing interesting, but it has a **mutating effect** on the argument list.

List Mutating Functions: Sorting

Similarly, we met a list-sorting function last time:

```
sorted : (list a) --> list a
```

This function *returns* a sorted version of the argument list, but leaves the argument list alone.

Python has another list-sorting function:

```
list.sort : (list a) --> NoneType
```

This function again *returns* nothing interesting, but it has a **mutating effect** on the argument list.

Often – but not always – argument-mutating functions return `None` because their purpose is to transform their arguments, not return results.

Adding Items to Lists

These list-mutating functions can be easily written using slice assignment:

inserts item into list at the specified index (shifting rightward):

```
list.insert(<list> , <index> , <item>)
```

inserts item at the end of list:

```
list.append(<list> , <item>)
```

inserts all items of another list at the end of this list:

```
list.extend(<this_list> , <another_list>)
```


Activity: List Mutating Functions 1

Use slice assignment to write a function `my_insert` that does what `list.insert` does:

```
def my_insert(items , index , item) :  
    """  
    signature: (list a , int , a) --> NoneType  
    inserts item at index of items list  
    """
```

For example:

```
> things = [1,2,3]  
> my_insert(things , 1 , 4)  
> things  
[1, 4, 2, 3]
```

Removing Items from Lists

These list-mutating functions can be easily written using slice assignment:

removes the item at index from list *and returns it*:

```
list.pop(<list> , <index>)
```

removes the first occurrence of item from list (it's an error if not present):

```
list.remove(<list> , <item>)
```

Activity: List Mutating Functions 2

Use slice assignment to write a function `my_pop` that does what `list.pop` does:

```
def my_pop(items , index) :  
    """  
    signature: (list a , int) --> a  
    removes element at index from items and returns it  
    """
```

For example:

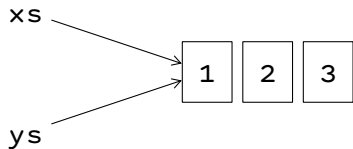
```
> things = [1,2,3]  
> my_pop(things , 1)  
2  
> things  
[1, 3]
```

Aliasing

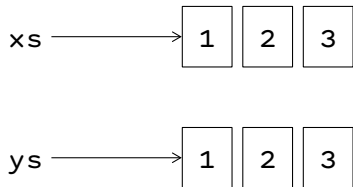
Because list contents can change over time, it is important to understand whether or not two things refer to the *same* list.

When they do, we call it **aliasing**, and any changes made through one alias will be visible through the other aliases.

```
xs = [1, 2, 3]
ys = xs                # aliased
list.reverse(xs)
```



```
xs = [1, 2, 3]
ys = list.copy(xs)    # independent
list.reverse(xs)
```



Detecting Aliasing

Without reading the documentation it is hard to know when data produced by built-in python functions share structure, but:

- Lists obtained by a variable assignment statement (`ys = xs`) *are aliased*.
- Lists obtained by the `list.copy` function *are independent*.
- Lists obtained by slicing another list (`ys = xs[i:j]`) *are independent*.

There is a predicate `_is_` that can tell you whether two things are aliases.

```
xs = [1, 2, 3]
ys = xs
zs = list.copy(xs)
xs is ys
xs is zs
xs == zs
```

To Do

Study for Midterm 1 in class next Tuesday, February 20
(practice tests available on Canvas and CodeRunner).

One index card or 1/2 page paper of notes allowed.

Bring your questions to the review session this Friday.

Finish *Homework 4: Strings and Lists* on CodeRunner by 11:00AM next Thursday.

Finish on *Project 1: The Turtle Module* on CodeRunner by next Friday.